JavaScript

Programación Web

JavaScript

- Es un lenguaje de programación interpretado, débilmente tipado y dinámico.
- Se puede programar utilizando varios paradigmas: programación dirigida por eventos (event-driven), funcional, orientada a objetos (basada en prototipos), imperativa y procedural.
- Originalmente su propósito era desarrollar código para los navegadores web, actualmente se puede utilizar para código en **servidores** y aplicaciones de **escritorio**. Un ambiente de ejecución muy popular es **Node.js**.
- El diseño de **JavaScript** es muy distinto al lenguaje **Java** aunque se parecen en el nombre.

JavaScript

- Ç

JavaScript

- DOM y Eventos
- HTTP Requests
- JSON
- SPA Single Page Applications

JavaScript

Todas las variables se definen utilizando cualquiera de estas tres palabras clave: var, let, const.

- **var** define una variable con un ámbito que abarca toda la función dónde fue definida. No utilizar en código moderno.
- **let** define una variable con un ámbito del bloque donde fue definido (como en c#).
- **const** define una variable que no se puede reasignar, la referencia es fija.

```
function myFunc(theObject) {
  theObject.make = "Toyota";
}

const myCar = {
  make: "Honda",
  model: "Accord",
  year: 1998,
};

console.log(myCar.make); // "Honda"
myFunc(myCar);
console.log(myCar.make); // "Toyota"
}
```

JavaScript

Las funciones son objetos de primera-clase.

- Se pueden asignar a variables.
- Se pueden pasar como parámetros o regresar como resultado de otra función.
- Pueden ser anónimas.
- Se pueden definir con la notación de flecha =>

```
const displayClosure = function() {
  let count = 0;
  return function () {
    return ++count;
  };
};

const inc = displayClosure();
```

JavaScript

```javascript
for (var i = 0; i < 3; i++) {
   setTimeout(() => console.log(i), 100);
}
```

```javascript
for (let i = 0; i < 3; i++) {
   setTimeout(() => console.log(i), 100);
}
```

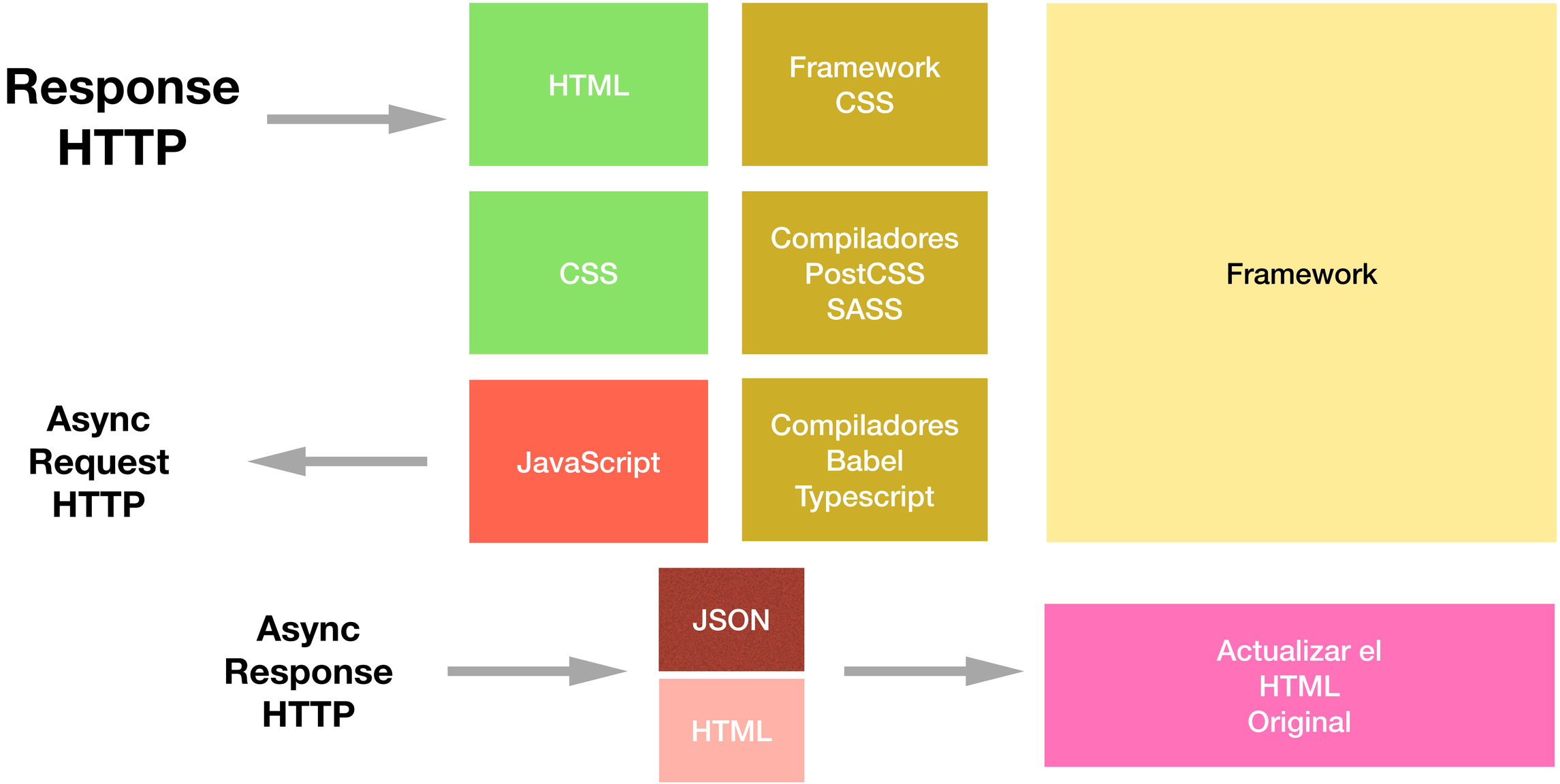
JavaScript

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Arquitectura Básica

(JS Frameworks)

Frameworks del lado del Cliente



Actualizar el DOM (HTML) con JSON

Ejemplo en *Vanilla* JavaScript

```
const state = [
  {
    id: "todo-0",
    name: "Learn some frameworks!",
  },
];
```

```
<ul>
  <li id="todo-0">
    <span>"Learn some frameworks!"</span>
    <button type="button">Delete</button>
  </li>
   …
</ul>
```

```
function buildTodoItemEl(id, name) {
  const item = document.createElement("li");
  const span = document.createElement("span");
  const textContent = document.createTextNode(name);

  span.appendChild(textContent);

  item.id = id;
  item.appendChild(span);
  item.appendChild(buildDeleteButtonEl(id));

  return item;
}
```

```
function renderTodoList() {
  const frag = document.createDocumentFragment();
  state.tasks.forEach((task) => {
    const item = buildTodoItemEl(task.id, task.name);
    frag.appendChild(item);
  });

  while (todoListEl.firstChild) {
    todoListEl.removeChild(todoListEl.firstChild);
  }
  todoListEl.appendChild(frag);
}
```

https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/Introduction

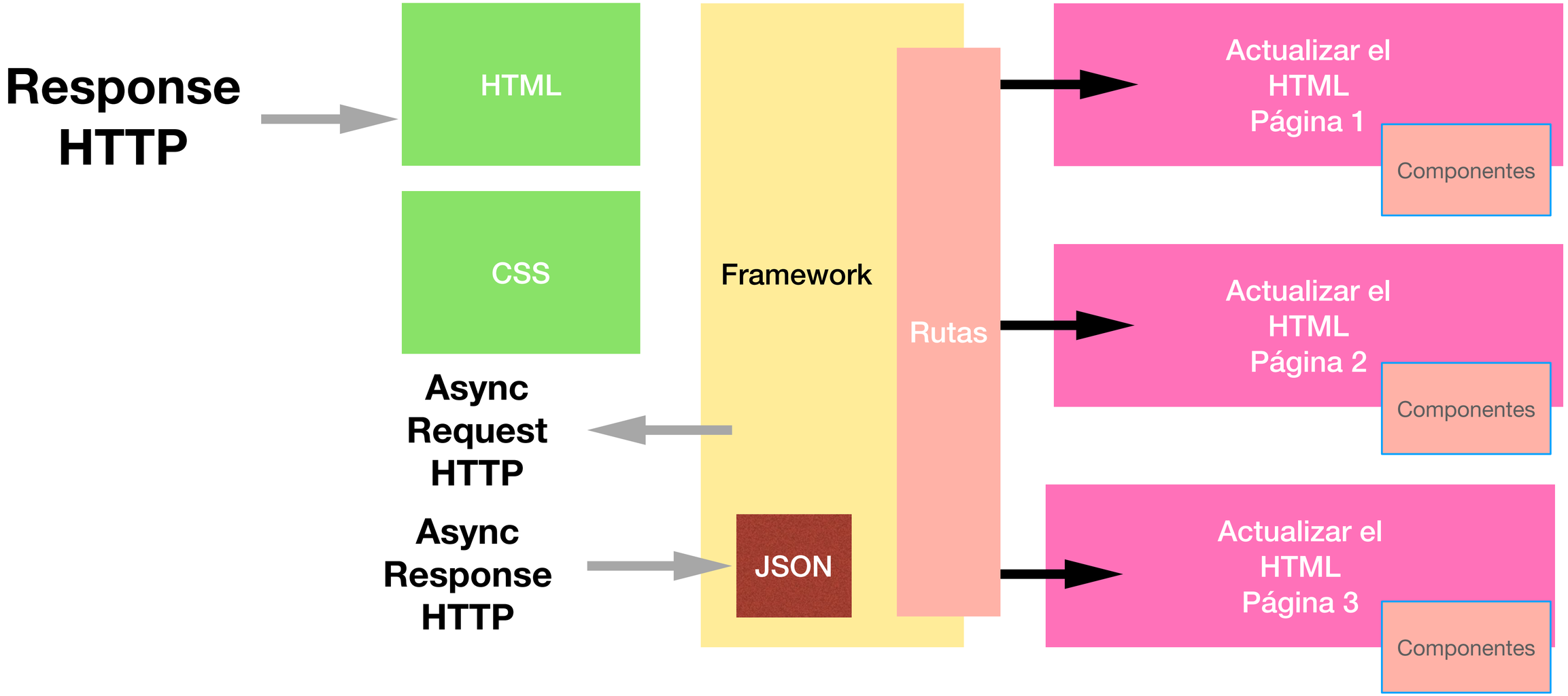
Actualizar el DOM (HTML) con JSON

Ejemplo en *Vue*

```
<ul>
  <li v-for="task in tasks" v-bind:key="task.id">
    <span>{{task.name}}</span>
    <button type="button">Delete</button>
  </li>
</ul>
```

https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/Introduction

Frameworks del lado del Cliente



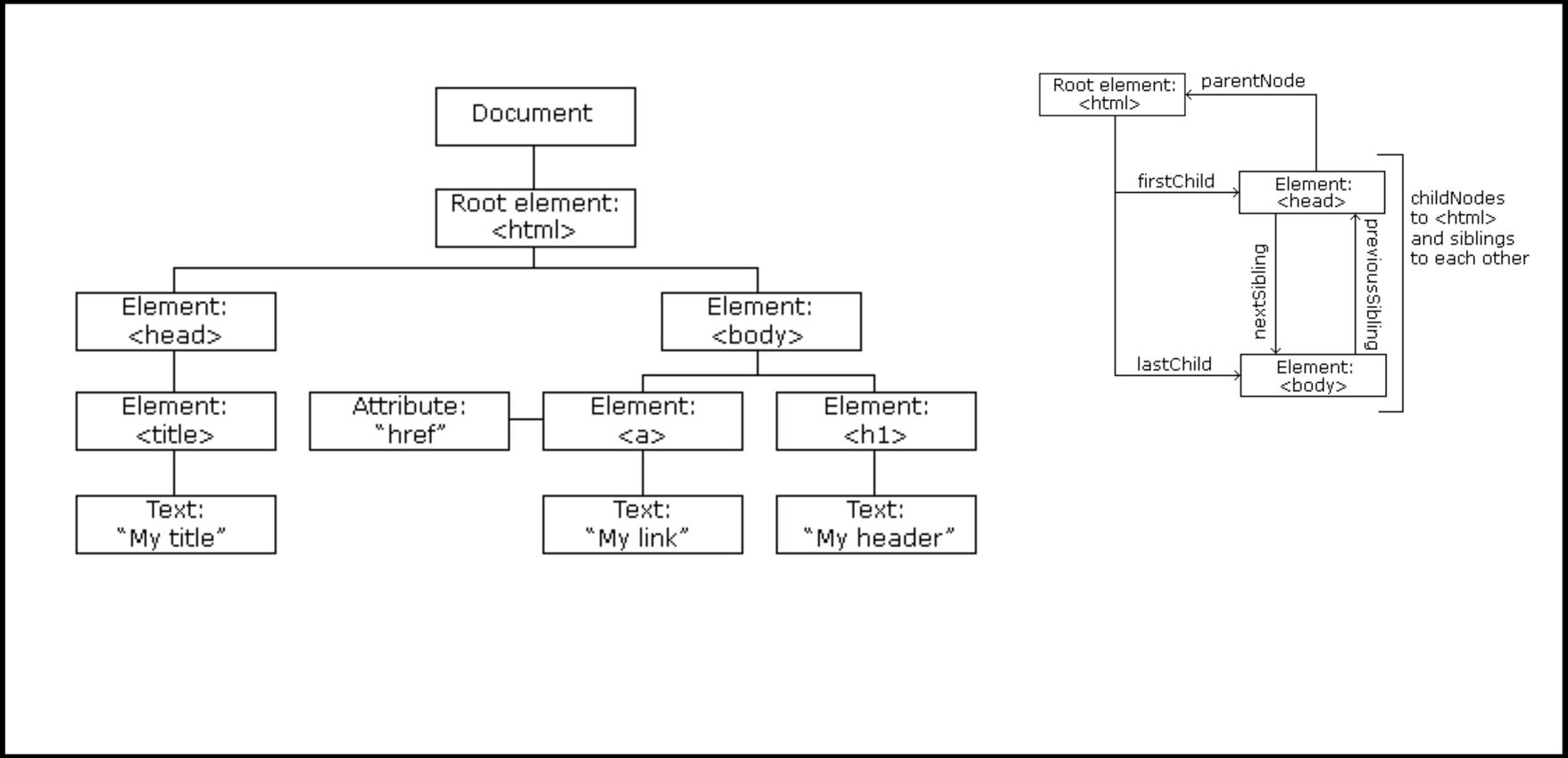
https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/Introduction

Frameworks del lado del Cliente

Framework	Browser support	Preferred DSL	Supported DSLs	Citation
Angular	Modern	TypeScript	HTML-based; TypeScript	official docs
React	Modern	JSX	JSX; TypeScript	official docs
Vue	Modern (IE9+ in Vue 2)	HTML-based	HTML-based, JSX, Pug	official docs
Ember	Modern (IE9+ in Ember version 2.18)	Handlebars	Handlebars, TypeScript	official docs

Manipulación del DOM

- document.querySelector('marca')
- document.querySelector('#id')
- document.querySelectorAll(‘.clase’)
- element.closest(selectors)
- element.matches(selector)
- element.firstElementChild( )
- element.nextElementSibling( )
- element.childNodes( )



https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

https://www.w3schools.com/js/js_htmldom_navigation.asp

Eventos

- Click
- Submit
- Scroll
- Drag
- DOMContentLoaded
- button.onclick = () => button.parentNode.remove();

HTTP Request

- fetch(url).then(r => r.json());

JSON

- Formato de Datos
- [{ 'url': '...' }] (objetos de JavaScript)
- data[0].url

Flujo Completo

Usuario → Evento → fetch → JSON → DOM